SOFTWARE ENGINEERING ACADEMIC PROJECT — MID-SEMESTER EVALUATION

Department of Computer Science & Engineering

Project Management Plan & Schedule

Project: AutoTestHub (Web Automation & Testing Platform)

AGILE SCRUM • SPRINT TIMELINE • RISK MANAGEMENT MATRIX

PROJECT PLAN METADATA

Methodology: Agile Scrum (2-week Sprints)

Total Project Duration: 16 Weeks (8 Sprints)

Current Milestone: Mid-Semester Evaluation (50% Complete)

Baseline Target: 40%–60% Completed

PROJECT ENGINEERING TEAM

Shristi Neekhra (Integration Lead & Scrum Master)

Shifa Siddiqui (Backend Architecture & Database)

Shruti Gupta (Selenium Engine & Worker Module)

Shriya Agarwal (Frontend Control Desk & Dashboard)

Supervisor: [Supervisor Name]

1. Methodology & Governance

AutoTestHub is managed using an adapted **Agile Scrum framework** organized into two-week sprints. The team conducts weekly standups, sprint reviews, and retrospective sessions to align module boundaries, review interface contracts, and verify automated test coverage.

1.1 Work Breakdown Structure (WBS)

WBS Level 1	WBS Level 2	Deliverable Component	Owner
1.0 Requirements & Architecture	1.1 Problem Analysis & Scope 1.2 Subsystem Interface Contract	SRS Document, IPC Specification	All Members
2.0 Presentation Tier	2.1 Control Desk Layout & Theme 2.2 Simulation Engine & Visualizer	HTML5/CSS3 UI, Live Log Streamer	Shriya Agarwal
3.0 Backend Services	3.1 Express Gateway & Routes 3.2 MongoDB Database Schemas	REST APIs, Mongoose Models	Shifa Siddiqui
4.0 Automation Engine	4.1 Chrome WebDriver Lifecycle 4.2 Dynamic Action Handlers	Python Runner, Explicit Waits	Shruti Gupta
5.0 Integration Pipeline	5.1 Asynchronous Execution Bridge 5.2 Process Spawner & Recovery	executionService.js, DB Persistence	Shristi Neekhra

2. Sprint Timeline & Milestone Schedule

Sprint / Phase	Calendar Weeks	Core Focus & Objectives	Status
Sprint 1: Inception & Scoping	Weeks 1 – 2	Problem research, stakeholder analysis, requirements definition, architecture selection.	COMPLETED
Sprint 2: Core Engine & Models	Weeks 3 – 4	Python Selenium worker prototype, Selenium Manager integration, Express skeleton, MongoDB schemas.	COMPLETED
Sprint 3: UI Desk & Integration	Weeks 5 – 6	Selenium Control Desk interface, execution service child_process bridge, unit & integration test suites.	COMPLETED
Milestone: Mid-Semester Evaluation	End of Week 6	Deliver working end-to-end pipeline, comprehensive reports, test evidence, and demo video (50% progress).	DELIVERED
Sprint 4: Advanced Assertions	Weeks 7 – 8	Element presence, text verification, attribute checks, and custom timeout configuration per action.	PLANNED
Sprint 5: React Dashboard	Weeks 9 – 10	Full React test builder UI, visual drag-and-drop action sequence designer, and run metrics dashboard.	PLANNED
Sprint 6: Distributed Queue	Weeks 11 – 12	Worker job queue (Redis / BullMQ) supporting concurrent execution across multiple headless workers.	PLANNED
Sprint 7: CI/CD & Export	Weeks 13 – 14	GitHub Actions workflow runner, Webhook triggers, PDF/HTML automated regression report generator.	PLANNED
Sprint 8: Hardening & Final Release	Weeks 15 – 16	Cross-browser testing (Firefox / Edge), performance benchmarking, final code freeze, and evaluation defense.	PLANNED

3. Risk Management & Mitigation Strategy

Risk Description	Severity / Likelihood	Technical Impact	Proactive Mitigation Strategy
Browser & Driver Mismatch	High / High	Browser version updates cause sudden ChromeDriver incompatibility and break test runs.	Adopted Selenium 4's built-in **Selenium Manager**, completely eliminating manual driver binaries and automating version resolution.
Asynchronous UI Timing Flakiness	High / Medium	Dynamic DOM updates cause false-negative element not found errors.	Banned arbitrary sleep timers; enforced `WebDriverWait` with `visibility_of_element_located` and `element_to_be_clickable`.
Orphaned Zombie Processes	High / Low	Unhandled exceptions leave Chrome processes running in the background, consuming RAM.	Encapsulated WebDriver lifecycles in strict `try–finally` blocks guaranteeing `driver.quit()` invocation even during unexpected fatal crashes.
Node-Python IPC Buffering Delays	Medium / Low	Node.js JSON parser chokes on verbose stdout logs emitted by third-party libraries.	Enforced stdout/stderr segregation (diagnostics to stderr) and implemented a robust regex JSON extractor (`/\{[\s\S]*\}/`) in Node.js.